

Taller Integrador ARSW 2026-I

Evolución de Arquitecturas Distribuidas con Java

Escuela Colombiana de Ingeniería Julio Garavito

Arquitecturas de Software - ARSW

Duración total: 8 horas - Dos sesiones de 4 horas

Modalidad: trabajo guiado, implementación progresiva y ejercicio
aplicado

Autor: Rodrigo Humberto Gualtero

Versión actualizada: comunicación distribuida, gRPC, microservicios y API Gateway

1. Introducción

Este taller integrador propone una ruta práctica para comprender la evolución de los mecanismos de comunicación entre aplicaciones distribuidas. A diferencia de un conjunto de ejercicios aislados, el taller desarrolla un mismo caso base y lo transforma progresivamente mediante distintos estilos arquitectónicos: cliente-servidor con sockets, HTTP, RMI, gRPC, microservicios y API Gateway.

La intención pedagógica es que el estudiante no solo copie código, sino que observe cómo cambia la arquitectura cuando cambian los mecanismos de comunicación, los contratos, las responsabilidades y el nivel de desacoplamiento entre componentes.

El taller está diseñado para desarrollarse en dos sesiones de cuatro horas. En cada estilo arquitectónico se presenta una guía paso a paso con un ejemplo corto, y luego se propone un ejercicio aplicado con mayor exigencia de análisis arquitectónico.

1.1 Objetivo general

Implementar y analizar la evolución de una aplicación distribuida en Java, identificando los problemas que resuelve cada estilo arquitectónico y las decisiones de diseño que aparecen al pasar de comunicación directa a servicios distribuidos, contratos formales, microservicios y un punto de entrada centralizado.

1.2 Resultados de aprendizaje

- Implementar una aplicación cliente-servidor usando sockets TCP.
- Exponer una funcionalidad básica mediante HTTP.
- Comprender el modelo RPC implementado con Java RMI.
- Definir contratos de comunicación usando Protocol Buffers y gRPC.
- Separar responsabilidades mediante microservicios pequeños y cohesivos.
- Centralizar el acceso a múltiples servicios usando un API Gateway sencillo.
- Comparar ventajas, limitaciones y trade-offs de cada estilo arquitectónico.

1.3 Organización de las sesiones

Sesión	Duración	Temas	Producto esperado
Sesión 1	4 horas	Sockets TCP, HTTP básico y RMI	Primeras tres versiones distribuidas del sistema base y ejercicios aplicados
Sesión 2	4 horas	gRPC, microservicios y API Gateway	Versión moderna orientada a servicios y diseño del ejercicio integrador

1.4 Caso base: Movie Information System

El caso base será un sistema mínimo de consulta de películas. El objetivo no es construir una aplicación cinematográfica completa, sino tener una funcionalidad sencilla que pueda evolucionar en varios estilos arquitectónicos.

ID	Título	Director	Año
1	Interstellar	Christopher Nolan	2014
2	Matrix	Wachowski	1999
3	Inception	Christopher Nolan	2010

Modelo conceptual inicial:

```
public class Movie {
    private int id;
    private String title;
    private String director;
    private int year;

    public Movie(int id, String title, String director, int year) {
        this.id = id;
        this.title = title;
        this.director = director;
        this.year = year;
    }

    public String toText() {
        return id + "," + title + "," + director + "," + year;
    }
}
```

1.5 Recomendaciones para el trabajo

- El trabajo debe desarrollarse de manera individual. Cada estudiante debe entregar su propia implementación, diagramas y reflexión arquitectónica.
- Ejecutar primero el ejemplo guiado antes de iniciar el ejercicio aplicado.
- Registrar evidencias de funcionamiento: capturas de consola, diagramas y respuestas obtenidas.
- Mantener los ejercicios sin base de datos; toda la información puede estar en memoria.
- Priorizar la comprensión arquitectónica sobre la cantidad de funcionalidades implementadas.

2. Parte I - Arquitectura Cliente-Servidor con Sockets TCP

2.1 ¿Qué problema resuelve este estilo?

La arquitectura cliente-servidor permite que un programa cliente solicite un servicio a un programa servidor que se encuentra escuchando en un puerto específico. En esta primera aproximación, el protocolo de comunicación será diseñado manualmente mediante mensajes de texto enviados por sockets TCP.

```
Cliente Java
|
| Mensaje de texto: MOVIE:1
v
Servidor TCP Java
|
| Respuesta: 1,Interstellar,Christopher Nolan,2014
v
Cliente muestra el resultado
```

Esta solución es útil para entender los fundamentos de comunicación distribuida, pero obliga al desarrollador a definir manualmente el formato de los mensajes, la validación, los errores y las respuestas.

2.2 Guía paso a paso: MovieServer TCP

Paso 1. Crear el proyecto

Cree una carpeta llamada movie-tcp y dentro de ella cree los archivos Movie.java, MovieRepository.java, MovieServer.java y MovieClient.java.

Paso 2. Crear el modelo Movie

```
public class Movie {
    private int id;
    private String title;
    private String director;
    private int year;

    public Movie(int id, String title, String director, int year) {
        this.id = id;
        this.title = title;
        this.director = director;
        this.year = year;
    }

    public int getId() {
        return id;
    }

    public String toText() {
        return id + "," + title + "," + director + "," + year;
    }
}
```

Paso 3. Crear el repositorio en memoria

```
import java.util.HashMap;
import java.util.Map;

public class MovieRepository {
    private Map<Integer, Movie> movies = new HashMap<>();

    public MovieRepository() {
        movies.put(1, new Movie(1, "Interstellar", "Christopher Nolan", 2014));
        movies.put(2, new Movie(2, "Matrix", "Wachowski", 1999));
        movies.put(3, new Movie(3, "Inception", "Christopher Nolan", 2010));
    }

    public Movie findById(int id) {
        return movies.get(id);
    }
}
```

```
}
```

Paso 4. Crear el servidor TCP

El servidor escuchará en el puerto 35000. Cuando reciba un mensaje con el formato MOVIE:id, buscará la película y responderá el resultado en texto plano.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class MovieServer {
    public static void main(String[] args) throws Exception {
        MovieRepository repository = new MovieRepository();
        ServerSocket serverSocket = new ServerSocket(35000);
        System.out.println("MovieServer TCP escuchando en puerto 35000...");

        while (true) {
            Socket clientSocket = serverSocket.accept();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(clientSocket.getInputStream()));
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);

            String request = in.readLine();
            String response = processRequest(request, repository);
            out.println(response);

            in.close();
            out.close();
            clientSocket.close();
        }
    }

    private static String processRequest(String request, MovieRepository repository) {
        if (request == null || !request.startsWith("MOVIE:")) {
            return "ERROR: formato inválido. Use MOVIE:id";
        }

        try {
            int id = Integer.parseInt(request.split(":")[1]);
            Movie movie = repository.findById(id);
            if (movie == null) {
                return "ERROR: película no encontrada";
            }
            return movie.toText();
        } catch (Exception e) {
            return "ERROR: solicitud inválida";
        }
    }
}
```

Paso 5. Crear el cliente TCP

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.Scanner;

public class MovieClient {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Ingrese el ID de la película: ");
        String id = scanner.nextLine();

        Socket socket = new Socket("127.0.0.1", 35000);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));

        out.println("MOVIE:" + id);
        String response = in.readLine();
        System.out.println("Respuesta del servidor: " + response);

        in.close();
        out.close();
        socket.close();
    }
}
```

```
}
```

Paso 6. Ejecutar

```
javac *.java  
java MovieServer  
# En otra terminal:  
java MovieClient
```

Prueba esperada: si el cliente envía el ID 1, el servidor debe responder: 1,Interstellar,Christopher Nolan,2014.

2.3 Ejercicio aplicado 1: Sistema de Gestión de Salones

Diseñe e implemente un servidor TCP para gestionar la reserva de salones de la Escuela. Este ejercicio debe reutilizar el estilo cliente-servidor, pero no debe copiar literalmente el caso de películas.

Requisitos funcionales

- El servidor debe mantener en memoria una lista inicial de salones: E301, E302, E303 y E304.
- Cada salón puede estar disponible o reservado.
- El cliente debe poder consultar el estado de un salón.
- El cliente debe poder reservar un salón disponible.
- El cliente debe poder liberar un salón reservado.

Protocolo mínimo sugerido

```
CONSULTAR_SALON,E303  
RESERVAR_SALON,E303  
LIBERAR_SALON,E303
```

Respuestas posibles

```
SALON_DISPONIBLE  
SALON_RESERVADO  
RESERVA_EXITOSA  
LIBERACION_EXITOSA  
ERROR_SALON_NO_EXISTE  
ERROR_OPERACION_INVALIDA
```

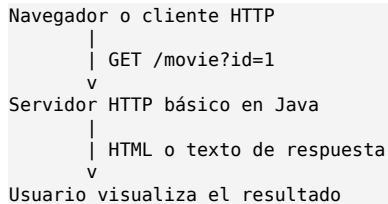
Preguntas de reflexión

- ¿Qué tan fácil sería agregar una nueva operación al protocolo?
- ¿Qué ocurre si dos clientes intentan reservar el mismo salón al mismo tiempo?
- ¿Dónde está definido realmente el contrato de comunicación: en un archivo formal o en convenciones de texto?

3. Parte II - Arquitectura HTTP con Java

3.1 ¿Qué problema resuelve este estilo?

La versión TCP obliga a escribir un cliente Java para consumir el servicio. Una evolución natural es exponer la funcionalidad mediante HTTP, de modo que pueda consultarse desde un navegador o desde herramientas como curl o Postman.



En este taller no se usará un framework web. El objetivo es observar cómo HTTP se construye sobre TCP y cómo aparecen conceptos como método, ruta, parámetro y respuesta.

3.2 Guía paso a paso: MovieHttpServer

Paso 1. Crear el servidor HTTP

Java incluye la clase `HttpServer` en el paquete `com.sun.net.httpserver`, suficiente para construir un servidor HTTP básico.

```
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class MovieHttpServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        MovieRepository repository = new MovieRepository();

        server.createContext("/movie", new MovieHandler(repository));
        server.setExecutor(null);
        server.start();
        System.out.println("MovieHttpServer escuchando en "
            + "http://localhost:8080/movie?id=1");
    }

    static class MovieHandler implements HttpHandler {
        private MovieRepository repository;

        public MovieHandler(MovieRepository repository) {
            this.repository = repository;
        }

        @Override
        public void handle(HttpExchange exchange) {
            try {
                String query = exchange.getRequestURI().getQuery();
                int id = extractId(query);
                Movie movie = repository.findById(id);

                String response;
                if (movie == null) {
                    response = "<html><body><h1>Película no encontrada</h1></body></html>";
                } else {
                    response = "<html><body><h1>" + movie.toText() + "</h1></body></html>";
                }

                exchange.sendResponseHeaders(200, response.getBytes().length);
                OutputStream os = exchange.getResponseBody();
                os.write(response.getBytes());
                os.close();
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}
```

```

        private int extractId(String query) {
            if (query == null || !query.startsWith("id=")) {
                return -1;
            }
            return Integer.parseInt(query.substring(3));
        }
    }
}

```

Paso 2. Ejecutar y probar

```

javac *.java
java MovieHttpServer

```

Luego abra en el navegador: <http://localhost:8080/movie?id=1>.

Paso 3. Analizar la diferencia con TCP

En TCP el contrato era una cadena como MOVIE:1. En HTTP la solicitud tiene una estructura más estándar: método, ruta, parámetros, encabezados y cuerpo. Esto facilita la interoperabilidad con clientes distintos a Java.

3.3 Ejercicio aplicado 2: Gestión de Salones vía HTTP

Transforme el ejercicio de gestión de salones para que funcione mediante HTTP. No es necesario construir una interfaz gráfica; puede probar con navegador, curl o Postman.

Rutas requeridas

```

GET /rooms
GET /rooms?id=E303
POST /rooms/reserve?id=E303
POST /rooms/release?id=E303

```

Respuesta esperada

Las respuestas pueden estar en texto plano o HTML simple. Lo importante es que el estudiante entienda cómo se traduce un protocolo textual propio a una interfaz HTTP con rutas más claras.

Preguntas de reflexión

- ¿Qué ventajas ofrece HTTP frente a un protocolo de texto definido manualmente?
- ¿Qué limitaciones tiene construir un servidor HTTP sin framework?
- ¿Cómo cambiaría esta solución si se usara JSON en lugar de HTML?

4. Parte III - RPC con Java RMI

4.1 ¿Qué problema resuelve este estilo?

RMI permite que un objeto en una máquina virtual de Java invoque métodos de un objeto ubicado en otra máquina virtual. Con RMI se deja de diseñar manualmente el formato de los mensajes, y la comunicación se expresa como invocación remota de métodos.



RMI es importante como tecnología histórica y conceptual, porque permite entender el modelo RPC. Sin embargo, está fuertemente asociado al ecosistema Java.

4.2 Guía paso a paso: MovieService con RMI

Paso 1. Crear una clase Movie serializable

```
import java.io.Serializable;

public class Movie implements Serializable {
    private int id;
    private String title;
    private String director;
    private int year;

    public Movie(int id, String title, String director, int year) {
        this.id = id;
        this.title = title;
        this.director = director;
        this.year = year;
    }

    @Override
    public String toString() {
        return id + " - " + title + " (" + year + ") - " + director;
    }
}
```

Paso 2. Crear la interfaz remota

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface MovieService extends Remote {
    Movie getMovie(int id) throws RemoteException;
}
```

Paso 3. Implementar el servicio

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.util.HashMap;
import java.util.Map;

public class MovieServiceImpl extends UnicastRemoteObject implements MovieService {
    private Map<Integer, Movie> movies = new HashMap<>();

    public MovieServiceImpl() throws RemoteException {
        movies.put(1, new Movie(1, "Interstellar", "Christopher Nolan", 2014));
        movies.put(2, new Movie(2, "Matrix", "Wachowski", 1999));
        movies.put(3, new Movie(3, "Inception", "Christopher Nolan", 2010));
    }
}
```

```

@Override
public Movie getMovie(int id) throws RemoteException {
    return movies.get(id);
}
}

```

Paso 4. Publicar el servicio

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class MovieRmiServer {
    public static void main(String[] args) throws Exception {
        MovieService service = new MovieServiceImpl();
        Registry registry = LocateRegistry.createRegistry(23000);
        registry.rebind("movieService", service);
        System.out.println("MovieService RMI publicado en puerto 23000...");
    }
}

```

Paso 5. Crear el cliente

```

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class MovieRmiClient {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.getRegistry("127.0.0.1", 23000);
        MovieService service = (MovieService) registry.lookup("movieService");
        Movie movie = service.getMovie(1);
        System.out.println("Película recibida: " + movie);
    }
}

```

Paso 6. Ejecutar

```

javac *.java
java MovieRmiServer
# En otra terminal:
java MovieRmiClient

```

4.3 Ejercicio aplicado 3: Inventario de Laboratorios

Implemente un sistema RMI para consultar y reservar equipos de laboratorio. El objetivo es diseñar una interfaz remota coherente y no depender de protocolos textuales.

Datos mínimos

- Código del equipo
- Nombre del equipo
- Laboratorio
- Estado: disponible o reservado

Métodos requeridos

```

List<String> consultarEquipos()
String consultarEquipo(String codigo)
boolean reservarEquipo(String codigo)
boolean liberarEquipo(String codigo)

```

Preguntas de reflexión

- ¿Qué cambió al pasar de HTTP a RMI?
- ¿Dónde está definido el contrato de comunicación?
- ¿Qué problemas tendría este sistema si un cliente no está escrito en Java?

5. Parte IV - Comunicación moderna con gRPC

5.1 ¿Qué problema resuelve este estilo?

gRPC permite implementar comunicación RPC moderna usando contratos definidos en archivos .proto. A diferencia de RMI, no está limitado a Java; un servicio puede estar escrito en Java y ser consumido por clientes en otros lenguajes. Además, los mensajes son fuertemente tipados y se serializan mediante Protocol Buffers.

```
movie.proto
|
| genera clases Java
v
Servidor gRPC <---- canal gRPC ----> Cliente gRPC
```

5.2 Guía paso a paso: MovieService con gRPC

Paso 1. Crear proyecto Maven

Cree un proyecto llamado movie-grpc con la siguiente estructura:

```
movie-grpc
├── pom.xml
├── src
│   └── main
│       ├── java
│       │   └── edu/eci/arsw/movie
│       └── proto
│           └── movie.proto
```

Paso 2. Configurar pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>edu.eci.arsw</groupId>
  <artifactId>movie-grpc</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

  <dependencies>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-netty-shaded</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-protobuf</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>io.grpc</groupId>
      <artifactId>grpc-stub</artifactId>
      <version>1.63.0</version>
    </dependency>
    <dependency>
      <groupId>com.google.protobuf</groupId>
      <artifactId>protobuf-java</artifactId>
      <version>3.25.3</version>
    </dependency>
    <dependency>
      <groupId>javax.annotation</groupId>
      <artifactId>javax.annotation-api</artifactId>
      <version>1.3.2</version>
    </dependency>
  </dependencies>

  <build>
```

```

<extensions>
  <extension>
    <groupId>kr.motd.maven</groupId>
    <artifactId>os-maven-plugin</artifactId>
    <version>1.7.1</version>
  </extension>
</extensions>
<plugins>
  <plugin>
    <groupId>org.xolstice.maven.plugins</groupId>
    <artifactId>protobuf-maven-plugin</artifactId>
    <version>0.6.1</version>
    <configuration>
<protocArtifact>com.google.protobuf:protoc:3.25.3:exe:${os.detected.classifier}</protocArtifact>
      <pluginId>grpc-java</pluginId>
      <pluginArtifact>io.grpc:protoc-gen-grpc-
        java:1.63.0:exe:${os.detected.classifier}</pluginArtifact>
    </configuration>
    <executions>
      <execution>
        <goals>
          <goal>compile</goal>
          <goal>compile-custom</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
</plugins>
</build>
</project>

```

Paso 3. Crear movie.proto

```

syntax = "proto3";

option java_multiple_files = true;
option java_package = "edu.eci.arsw.movie";
option java_outer_classname = "MovieProto";

service MovieService {
  rpc GetMovie (MovieRequest) returns (MovieResponse);
}

message MovieRequest {
  int32 id = 1;
}

message MovieResponse {
  int32 id = 1;
  string title = 2;
  string director = 3;
  int32 year = 4;
  bool found = 5;
}

```

Paso 4. Generar clases

mvn clean compile

Después de ejecutar Maven, revise la carpeta target/generated-sources. Allí estarán las clases generadas a partir del contrato .proto.

Paso 5. Implementar el servidor gRPC

```

package edu.eci.arsw.movie;

import io.grpc.Server;
import io.grpc.ServerBuilder;
import io.grpc.stub.StreamObserver;
import java.util.HashMap;
import java.util.Map;

public class MovieGrpcServer {
  public static void main(String[] args) throws Exception {
    Server server = ServerBuilder.forPort(50051)
      .addService(new MovieServiceImpl())
      .build();

    server.start();
    System.out.println("Movie gRPC Server iniciado en puerto 50051");
  }
}

```

```

        server.awaitTermination();
    }

    static class MovieServiceImpl extends MovieServiceGrpc.MovieServiceImplBase {
        private Map<Integer, MovieResponse> movies = new HashMap<>();

        public MovieServiceImpl() {
            movies.put(1, MovieResponse.newBuilder()
                .setId(1).setTitle("Interstellar")
                .setDirector("Christopher Nolan").setYear(2014).setFound(true).build());
            movies.put(2, MovieResponse.newBuilder()
                .setId(2).setTitle("Matrix")
                .setDirector("Wachowski").setYear(1999).setFound(true).build());
            movies.put(3, MovieResponse.newBuilder()
                .setId(3).setTitle("Inception")
                .setDirector("Christopher Nolan").setYear(2010).setFound(true).build());
        }

        @Override
        public void getMovie(MovieRequest request,
            StreamObserver<MovieResponse> responseObserver) {
            MovieResponse response = movies.get(request.getId());
            if (response == null) {
                response = MovieResponse.newBuilder()
                    .setId(request.getId())
                    .setFound(false)
                    .build();
            }
            responseObserver.onNext(response);
            responseObserver.onCompleted();
        }
    }
}

```

Paso 6. Implementar el cliente gRPC

```

package edu.eci.arsw.movie;

import io.grpc.ManagedChannel;
import io.grpc.ManagedChannelBuilder;

public class MovieGrpcClient {
    public static void main(String[] args) {
        ManagedChannel channel = ManagedChannelBuilder
            .forAddress("localhost", 50051)
            .usePlaintext()
            .build();

        MovieServiceGrpc.MovieServiceBlockingStub stub =
            MovieServiceGrpc.newBlockingStub(channel);

        MovieRequest request = MovieRequest.newBuilder()
            .setId(1)
            .build();

        MovieResponse response = stub.getMovie(request);

        if (response.getFound()) {
            System.out.println("Película: " + response.getTitle()
                + " - " + response.getDirector()
                + " - " + response.getYear());
        } else {
            System.out.println("Película no encontrada");
        }

        channel.shutdown();
    }
}

```

Paso 7. Ejecutar

```

mvn clean compile
mvn exec:java -Dexec.mainClass="edu.eci.arsw.movie.MovieGrpcServer"
# En otra terminal:
mvn exec:java -Dexec.mainClass="edu.eci.arsw.movie.MovieGrpcClient"

```

5.3 Ejercicio aplicado 4: Sistema de Bienestar Universitario con gRPC

Diseñe e implemente un servicio gRPC para gestionar solicitudes de citas de bienestar universitario. Este ejercicio evalúa la capacidad de modelar contratos y no solamente de modificar nombres de clases.

Servicio requerido

```
service AppointmentService {  
  rpc RequestAppointment (AppointmentRequest) returns (AppointmentResponse);  
  rpc CancelAppointment (CancelRequest) returns (CancelResponse);  
  rpc GetAppointments (StudentRequest) returns (AppointmentList);  
}
```

Entidades mínimas

- Student: id, name, institutionalEmail.
- Appointment: id, studentId, serviceType, date, status.
- ServiceType: MEDICINE, PSYCHOLOGY, DENTISTRY.
- Status: REQUESTED, CANCELLED, ATTENDED.

Reglas

- Una cita solicitada debe quedar en estado REQUESTED.
- Una cita cancelada no debe aparecer como activa.
- El sistema debe permitir consultar las citas de un estudiante.
- La información se debe mantener en memoria.

Preguntas de reflexión

- ¿Por qué el archivo .proto se considera un contrato?
- ¿Qué tan fácil sería crear un cliente en otro lenguaje?
- ¿Qué diferencias encuentra entre RMI y gRPC?

6. Parte V - Arquitectura de Microservicios

6.1 ¿Qué problema resuelve este estilo?

Hasta este punto, aunque se ha cambiado la tecnología de comunicación, el sistema sigue concentrando toda la responsabilidad en un único servicio. Cuando un sistema crece, es común separar responsabilidades en servicios más pequeños, autónomos y altamente cohesivos.

```
Cliente
|
+---- MovieService      (películas)
+---- ReviewService     (reseñas)
+---- RecommendationService (recomendaciones)
```

La arquitectura de microservicios no significa crear muchos servicios sin criterio. Cada servicio debe tener una responsabilidad clara y debe evitar conocer detalles internos de otros servicios.

6.2 Guía paso a paso: dividir el sistema de películas

Paso 1. Identificar responsabilidades

Servicio	Responsabilidad	Puerto sugerido
MovieService	Consultar información de películas	50051
ReviewService	Consultar reseñas asociadas a una película	50052
RecommendationService	Sugerir películas relacionadas	50053

Paso 2. Definir contrato de ReviewService

```
syntax = "proto3";
option java_multiple_files = true;
option java_package = "edu.eci.arsw.review";

service ReviewService {
  rpc GetReviews (ReviewRequest) returns (ReviewList);
}

message ReviewRequest {
  int32 movieId = 1;
}

message Review {
  string author = 1;
  string comment = 2;
  int32 rating = 3;
}

message ReviewList {
  repeated Review reviews = 1;
}
```

Paso 3. Definir contrato de RecommendationService

```
syntax = "proto3";
option java_multiple_files = true;
option java_package = "edu.eci.arsw.recommendation";

service RecommendationService {
  rpc GetRecommendations (RecommendationRequest) returns (RecommendationList);
}

message RecommendationRequest {
  int32 movieId = 1;
}

message RecommendationList {
  repeated string titles = 1;
}
```

Paso 4. Ejecutar servicios en puertos distintos

Cada servicio debe tener su propio servidor gRPC. El código será similar al MovieGrpcServer, pero cada servidor debe publicar una responsabilidad diferente y escuchar en un puerto diferente.

```
Server movieServer = ServerBuilder.forPort(50051)
    .addService(new MovieServiceImpl())
    .build();

Server reviewServer = ServerBuilder.forPort(50052)
    .addService(new ReviewServiceImpl())
    .build();

Server recommendationServer = ServerBuilder.forPort(50053)
    .addService(new RecommendationServiceImpl())
    .build();
```

Paso 5. Crear un cliente que consulte varios servicios

El cliente puede abrir un canal para cada servicio. Esto funciona, pero genera un nuevo problema: el cliente empieza a conocer demasiados servicios y demasiados puertos.

```
MovieClient
|---- localhost:50051 -> MovieService
|---- localhost:50052 -> ReviewService
|---- localhost:50053 -> RecommendationService
```

6.3 Ejercicio aplicado 5: Descomposición de Bienestar Universitario

A partir del ejercicio gRPC de citas de bienestar, proponga e implemente una descomposición inicial en microservicios. La solución debe priorizar claridad arquitectónica, no cantidad de líneas de código.

Servicios mínimos sugeridos

Servicio	Responsabilidad
AppointmentService	Gestionar citas y turnos de atención.
MedicalService	Gestionar información básica de especialidades médicas disponibles.
GymService	Gestionar reservas simples de sesiones de gimnasio.
RecreationService	Gestionar préstamo o reserva de recursos recreativos.

Producto esperado

- Diagrama de microservicios.
- Descripción de la responsabilidad de cada servicio.
- Al menos dos servicios implementados y ejecutándose en puertos distintos.
- Cliente que consuma directamente los servicios implementados.

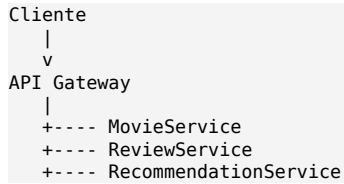
Preguntas de reflexión

- ¿Por qué decidió separar esos servicios y no otros?
- ¿Qué datos pertenecen a cada servicio?
- ¿Qué riesgo aparece cuando el cliente conoce todos los servicios?

7. Parte VI - API Gateway

7.1 ¿Qué problema resuelve este estilo?

Cuando el cliente conoce todos los microservicios, queda acoplado a sus direcciones, puertos y contratos individuales. Un API Gateway centraliza el acceso y actúa como punto de entrada único hacia el sistema.



En este taller el Gateway se implementará como un programa Java sencillo. No se busca usar infraestructura avanzada, sino comprender el patrón arquitectónico.

7.2 Guía paso a paso: MovieGateway

Paso 1. Definir responsabilidades del Gateway

- Recibir solicitudes del cliente.
- Conectarse internamente a los servicios necesarios.
- Unificar la respuesta.
- Evitar que el cliente conozca los puertos de cada microservicio.

Paso 2. Crear un Gateway de consola

Para mantener el taller en 8 horas, el Gateway puede implementarse como una aplicación Java que se comunica por gRPC con los servicios internos y presenta una interfaz unificada en consola.

```

public class MovieGateway {
    public static void main(String[] args) {
        // 1. Crear canal hacia MovieService
        // 2. Crear canal hacia ReviewService
        // 3. Crear canal hacia RecommendationService
        // 4. Consultar los servicios internos
        // 5. Construir una respuesta integrada para el cliente
    }
}

```

Paso 3. Forma esperada de la respuesta

Película: Interstellar
Director: Christopher Nolan
Año: 2014

Reseñas:
- Excelente película de ciencia ficción. Rating: 5
- Visualmente impresionante. Rating: 4

Recomendaciones:
- Inception
- Contact
- 2001: A Space Odyssey

Paso 4. Análisis

Aunque el Gateway centraliza el acceso, también puede convertirse en un punto crítico. Si el Gateway falla, el cliente pierde acceso a todo el sistema. Por eso, en arquitecturas reales se requieren estrategias de disponibilidad, monitoreo y escalabilidad.

7.3 Ejercicio aplicado 6: WellnessGateway

Construya un Gateway para centralizar el acceso a los servicios del sistema de bienestar universitario.

Servicios internos

- AppointmentService
- MedicalService
- GymService
- RecreationService

Operaciones mínimas del Gateway

```
requestAppointment(studentId, serviceType)
getStudentWellnessSummary(studentId)
reserveGymSession(studentId, timeSlot)
reserveRecreationResource(studentId, resourceId)
```

Preguntas de reflexión

- ¿Qué simplifica el Gateway para el cliente?
- ¿Qué complejidad agrega al sistema?
- ¿Qué pasaría si el Gateway empieza a contener demasiada lógica de negocio?

8. Ejercicio integrador final - Plataforma ECICIENCIA

Como cierre del taller, los estudiantes deben diseñar la arquitectura de una plataforma distribuida para apoyar la gestión del evento ECICIENCIA. Este ejercicio no exige implementar todo el sistema, pero sí requiere justificar decisiones arquitectónicas usando los estilos trabajados durante el taller.

8.1 Contexto

La Escuela necesita una plataforma para organizar actividades académicas, talleres, charlas y experiencias tecnológicas durante ECICIENCIA. El sistema debe permitir registrar asistentes, consultar la agenda, reservar talleres y controlar el aforo de cada actividad.

8.2 Funcionalidades mínimas

- Registro de asistentes.
- Consulta de agenda.
- Reserva de cupos en talleres.
- Control de aforo por actividad.
- Consulta de actividades por franja horaria.

8.3 Actividades del ejercicio

- Identifique los microservicios necesarios.
- Defina la responsabilidad de cada microservicio.
- Proponga los contratos gRPC principales.
- Diseñe un API Gateway para centralizar el acceso.
- Elabore un diagrama de arquitectura.
- Justifique por qué no usaría un único servicio monolítico para todo.

8.4 Entregable del ejercicio final

- Diagrama arquitectónico.
- Lista de servicios y responsabilidades.
- Al menos un archivo .proto propuesto.
- Descripción del Gateway.
- Reflexión de máximo una página sobre la evolución arquitectónica del taller.

9. Comparación de estilos arquitectónicos

Estilo	Ventaja principal	Limitación principal	Cuándo usarlo
Sockets TCP	Control total sobre la comunicación	Protocolo manual y bajo nivel	Aprendizaje de redes o protocolos muy específicos
HTTP	Interoperabilidad y acceso desde navegador	Requiere diseñar rutas y formatos de respuesta	APIs web simples
RMI	Invocación remota de métodos en Java	Dependencia fuerte del ecosistema Java	Sistemas Java distribuidos controlados
gRPC	Contratos formales, eficiencia e interoperabilidad	Requiere configuración de protobuf	Microservicios y comunicación interna
Microservicios	Separación de responsabilidades y escalabilidad	Mayor complejidad operacional	Sistemas con dominios claramente separables
API Gateway	Punto de entrada único	Puede convertirse en cuello de botella	Sistemas con varios servicios internos

10. Rúbrica de evaluación

Criterio	Porcentaje	Descripción
Implementación técnica	30%	Los ejemplos y ejercicios ejecutan correctamente según el estilo arquitectónico solicitado.
Diseño arquitectónico	25%	La separación de responsabilidades, contratos y servicios es coherente.
Análisis y reflexión	20%	El estudiante explica ventajas, limitaciones y trade-offs de cada estilo.
Diagramas	15%	Los diagramas representan correctamente componentes, comunicaciones y responsabilidades.
Orden y documentación	10%	El código está organizado y el entregable es claro.

11. Entregables del taller

- Código fuente de las versiones implementadas.
- Evidencias de ejecución de cada parte.
- Diagramas de arquitectura por estilo.
- Solución de los ejercicios aplicados.
- Diseño del ejercicio integrador ECICIENCIA.
- Reflexión final sobre la evolución de la arquitectura.

12. Cierre

Este taller muestra que las arquitecturas distribuidas no aparecen por moda, sino como respuesta a problemas concretos: comunicación remota, interoperabilidad, contratos, separación de responsabilidades, escalabilidad y reducción del acoplamiento. El valor del ejercicio no está solo en ejecutar código, sino en comprender por qué cada estilo arquitectónico existe y qué problema intenta

resolver.

Al finalizar, el estudiante debe estar en capacidad de explicar cómo una solución simple puede evolucionar desde un servidor TCP hasta una arquitectura basada en servicios y Gateway, reconociendo los costos y beneficios de cada decisión.